Most Commonly Used 10 DAX Functions in Power BI

analyticsvidhya.com/blog/2024/07/dax-functions-in-power-bi

Yana Khare July 15, 2024

Power BI uses a set of functions, operators, and constants called DAX to perform dynamic computations and analysis. One can enhance their Power BI competency by using DAX features that help in **data modeling** and reporting. This article examines the top DAX features that any Power BI user should know.

Free Certification Courses

What is Power BI?

Power BI was by Microsoft. It is a business analytics software that enables clients to analyze, view, and share their data. Power BI has ‘wizards’ that are used to create interfaces that can easily manipulate the data. It also provides **business intelligence** and tools such as graphs and charts through which end users can generate reports and dashboards.

What are DAX Functions?

Some commonly used operators, functions, and constants used in formulae and expressions in Power BI, Power Pivot, and Analysis Services included DAX, which can be used to calculate and even return outcomes.

Among the main attributes of DAX functions are:

- **Data Aggregation:** Data may be summarized using functions like SUM, AVERAGE, COUNT, and SUMX.
- **Filtering:** Complex data filters may be defined using functions like FILTER and CALCULATE.
- **Time Intelligence:** Date and time computations are handled by functions like DATEADD, DATEDIFF, and YEAR.
- **Lookup functions:** LOOKUPVALUE and other similar functions facilitate the retrieval of relevant data from many tables.

Why is Power BI Essential for Data Analysis?

Power BI is essential for data analysis due to the following reasons:

1. **Interactive Visualizations**: A vast array of dynamic and adaptable **data visualizations** offered by Power BI facilitate data interpretation and meaningful presentation.
2. **Real-time Data Access**: It facilitates the connectivity of many data sources, including real-time data streams, so that analysis and reporting are up to date.
3. **Data Transformation and Preparation:** Power BI offers an easy way to clean up, transform, and prepare data for analysis in Power Query so users can write a little code.
4. **Advanced Analytics:** Power BI provides for complex calculations and rich analyses. The DAX features an AI built into the system to help the users.
5. **Integration and Collaboration:** Power BI is compatible with Microsoft Excel, Azure, and SharePoint tools. It also enhances teamwork and sharing of different tasks and ideas.

Top DAX Functions in Power BI

Here are some of the most commonly used DAX functions:

1. CALCULATE
2. SUM and SUMX
3. CALCULATETABLE
4. RANKX
5. DATEDIFF
6. DATEADD
7. COUNT and COUNTROWS
8. LOOKUPVALUE

9. FILTER
10. RELATED

Below is a sample dataset of the Sales and Products tables that can be used to perform the DAX functions mentioned above.

Sales Table

Products Table

1. CALCULATE

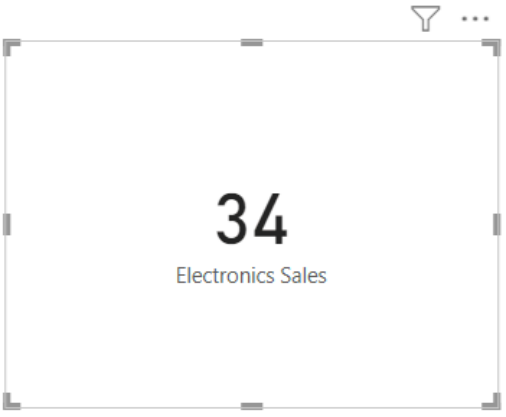
The CALCULATE function is among the strongest DAX formula. It evaluates an expression in the context of a changed filter. With CALCULATE, you may alter the environment in which data is filtered, enabling more adaptable and dynamic computations.

Syntax:

```
CALCULATE(<expression>, <filter1>, <filter2>...)
```

Example:

```
Electronics Sales =
CALCULATE(
    SUM(Sales[Quantity]),
    Products[Category] = "Electronics"
)
```



Explanation:

This measure calculates the total sales for electronics products only.

- The expression `SUM(Sales[Quantity] * Sales[UnitPrice])` calculates the total sales.
- The filter `Products[Category] = "Electronics"` modifies the context to include only products in the Electronics category.
- CALCULATE applies this filter, overriding any existing filters on the Products table.

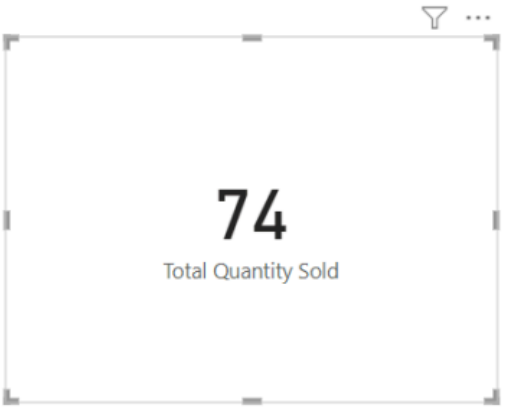
2. SUM and SUMX

The SUM function is straightforward, summing up all values in a column. SUMX, on the other hand, is an iterator function that sums up the results of an expression evaluated for each row in a table.

```
SUM(<column>)
SUMX(<table>, <expression>)

Total Quantity Sold = SUM(Sales[Quantity])

Total Profit =
SUMX(
    Sales,
    Sales[Quantity] * (Sales[UnitPrice] - RELATED(Products[Cost]))
)
```



```
Total Profit =
SUMX(
    Sales,
    Sales[Quantity] * (Sales[UnitPrice] - RELATED(Products[Cost]))
)
```

SUM operates on a single column or on an expression that results in a column of values. It adds up all the values in that column, respecting any filter context that's in place. It's efficient for simple summations but limited when you need row-by-row calculations involving multiple columns or related tables.

SUMX iterates over each row in the specified table (Sales in this case). For each row, it evaluates the given expression: `Sales[Quantity] * (Sales[UnitPrice] - RELATED(Products[Cost]))`

- It multiplies the quantity sold by the difference between the unit price and the product cost.
- RELATED(Products[Cost]) brings in the cost from the Products table for each product.

After calculating this for each row, SUMX sums up all the results.

3. CALCULATETABLE

CALCULATETABLE evaluates a table expression in a modified filter context, similar to how CALCULATE works with scalar expressions.

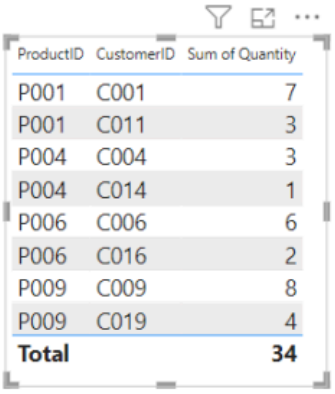
Syntax:

```
CALCULATETABLE(<table_expression>, <filter1>, <filter2>,...)
```

Example:

```
Electronics_Sales =CALCULATETABLE(Sales,Products[Category] = "Electronics")
```

```
Electronics Sales =
CALCULATETABLE(
Sales,
Products[Category] = "Electronics"
)
```



ProductID	CustomerID	Sum of Quantity
P001	C001	7
P001	C011	3
P004	C004	3
P004	C014	1
P006	C006	6
P006	C016	2
P009	C009	8
P009	C019	4
Total		**34**

Explanation:

- It starts with the entire Sales table.
- It then applies the filter `Products[Category] = "Electronics"`.
- This filter is applied through the relationship between Sales and Products tables. The result is a subset of the Sales table containing only sales of electronic products.
- This function is handy for creating virtual tables filtered according to specific criteria.

4. RANKX

RANKX ranks items in a table based on an expression. It helps create rankings within a data set.

Syntax:

```
Rankx(<table>,<expression>,<value>(optional),<order>(optional),<ties>(optional))
```

Example:

```
RANKX(
    ALL(Sales[SalesPersonID]),
    CALCULATE(SUM(Sales[Quantity] * Sales[UnitPrice]))
)
```

Explanation:

- The expression CALCULATE(SUM(Sales[Quantity] * Sales[UnitPrice])) calculates the total sales amount for each salesperson.
- RANKX evaluates the expression for each row in the specified table.
- It then assigns a rank based on the result of that expression.

- By default, RANKX assigns a lower rank (closer to 1) for higher values.

In Power BI, RANKX helps in generating rankings dynamically within reports.

5. DATEDIFF

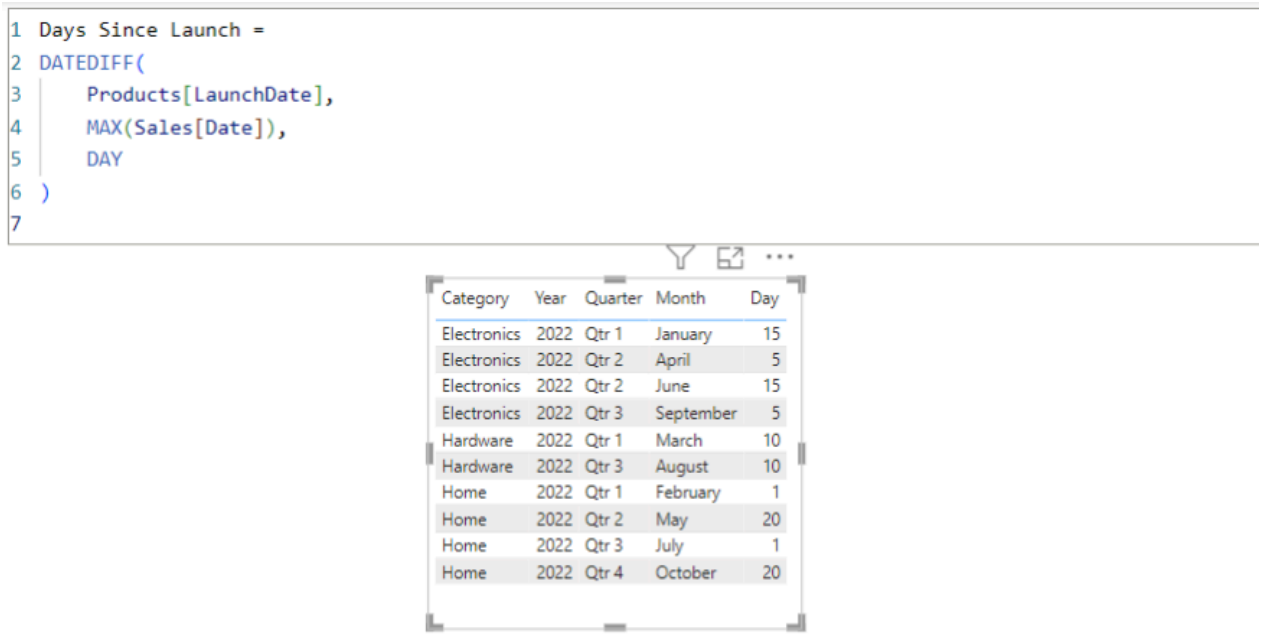
The DATEDIFF function calculates the difference between two dates.

Syntax:

```
Duration = DATEDIFF(StartDate, EndDate, DAY)
```

Example

```
Days Since Launch =
DATEDIFF(
    Products[LaunchDate],
    MAX(Sales[Date]),
    DAY
)
```



Category	Year	Quarter	Month	Day
Electronics	2022	Qtr 1	January	15
Electronics	2022	Qtr 2	April	5
Electronics	2022	Qtr 2	June	15
Electronics	2022	Qtr 3	September	5
Hardware	2022	Qtr 1	March	10
Hardware	2022	Qtr 3	August	10
Home	2022	Qtr 1	February	1
Home	2022	Qtr 2	May	20
Home	2022	Qtr 3	July	1
Home	2022	Qtr 4	October	20

Explanation:

This measure calculates the number of days between each product's launch date and the latest sale date.

- Products[LaunchDate] is the start date.
- MAX(Sales[Date]) finds the latest sale date in the current context.
- DAY specifies that the difference should be calculated in days.

In Power BI, DATEDIFF helps calculate the time elapsed between two dates in various units (days, months, years).

6. DATEADD

DATEADD shifts a date by a specified number of intervals. It helps create comparisons over time.

Syntax:

```
DATEADD(<dates>, <number_of_intervals>, <interval>)
```

Example:

```
NewLaunchDate = DATEADD(Products[LaunchDate], 1, MONTH)
```

Explanation:

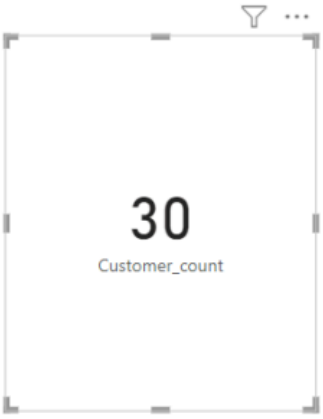
In Power BI, DATEADD enables time intelligence operations, such as comparing the current month's data to the previous month's.

7. COUNT and COUNTROWS

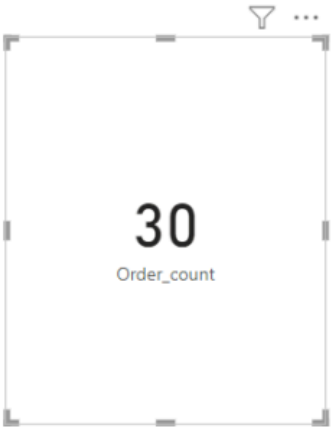
While COUNTROWS counts the number of rows in a table, COUNT counts the number of non-empty values in a column.

Example:

```
CustomerCount = COUNT(Sales[CustomerID])
OrderCount = COUNTROWS(Sales)
```



```
1 Order_count = COUNTROWS(Sales)
```



Explanation:

- **COUNT** on CustomerID column: This will count the number of non-blank entries in the CustomerID column.
- **COUNTROWS** on the entire Sales table: This will count the total number of rows in the table.

In Power BI, these functions help determine the size of a dataset or subset.

8. LOOKUPVALUE

LOOKUPVALUE retrieves the value of a column in a table, akin to a **VLOOKUP in Excel**.

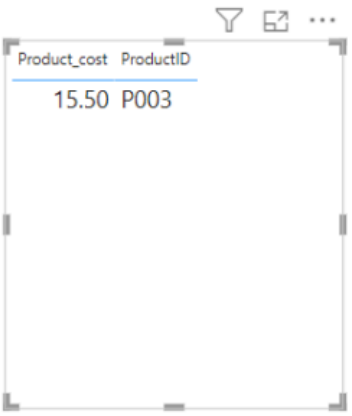
Syntax:

```
LOOKUPVALUE(<result_column>, <search_column1>, <search_value1> [,
<search_column2>, <search_value2> [, … ] ] )
```

Example:

```
ProductCost = LOOKUPVALUE(Products[Cost], Products[ProductID], "P003")
```

```
1 Product_cost = LOOKUPVALUE(Products[Cost],Products[ProductID],"P003")
```



Explanation:

- **`result_column`**: The column from which you want to return the value.
- **`search_column1`**: The column where you search for the value.
- **`search_value1`**: The value you want to find in the `search_column1`.
- This formula searches the `Products` table for the `ProductID` “P003” and returns the corresponding `Cost`.

In Power BI, LOOKUPVALUE is essential for fetching related data from different tables.

9. FILTER

The FILTER function returns a table representing a subset of another table filtered by a given expression.

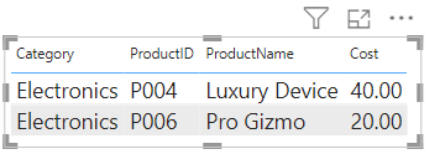
Syntax:

```
FILTER(<table>,<filter>)
```

Example:

```
FilteredProducts = FILTER(
    Products,
    Products[Category] = "Electronics" && Products[Cost] > 10
)
```

```
1 filtered_table = FILTER(Products,Products[Category]="Electronics" && Products[Cost]>10)
```



Category	ProductID	ProductName	Cost
Electronics	P004	Luxury Device	40.00
Electronics	P006	Pro Gizmo	20.00

Explanation:

This condition consists of two parts combined with the logical AND operator (`&&`):

- `Products[Category] = "Electronics"`: This part of the condition filters the rows where the `Category` is “Electronics”.
- `Products[Cost] > 10`: This part of the condition further filters those rows where the `Cost` is greater than 10.

In Power BI, FILTER is used to narrow down data based on specific conditions.

10. RELATED

The RELATED function fetches a related value from another table, leveraging the relationships between tables.

Syntax:

```
RELATED(<Column>)
```

Example:

```
Category Description = RELATED(Categories[Description])
```

Explanation:

This DAX expression fetches the `Description` from the `Categories` table for each row in the `Products` table based on the relationship defined between the `Category` columns of both tables.

In Power BI, RELATED is crucial for retrieving data from related tables.

Conclusion

Proficiency with these Power BI DAX functions will significantly improve your data analysis and modeling capacity. Understanding and efficiently using functions like CALCULATE, SUMX, RANKX, DATEDIFF, and LOOKUPVALUE will enable you to perform intricate computations and produce dynamic, informative results. The DAX functions in Power BI provide the tools to transform unstructured data into insightful information, whether doing total calculations, rating data, or working with date values.

If you want to learn about DAX for Power BI in-depth, here is a YouTube tutorial for you: Hands-on with Data Analysis Expressions (DAX) for Power BI

Microsoft Excel: Formulas & Functions